

박수민_DS1302과제

1. 전역변수 -> 지역변수
2. 시각보정기능

```
#main.c
...

/*
 * 10.MY_DS1302.c
 *
 * Created: 2026-06-29 오전 9:52:33
 * Author : kccistc
 */

/*
 * 10.DS1302.c
 *
 * Created: 2026-06-26 오후 2:39:07
 * Author : kccistc
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h> // sei, ISR 등
#include <stdio.h>
#include <util/delay.h>

extern void init_uart0();
extern void UART0_transmit(uint8_t data);
// extern void dht11_main();
extern void ds1302_main();

FILE OUTPUT = FDEV_SETUP_STREAM(UART0_transmit, NULL, _FDEV_SETUP_WRITE);

int main(void)
{
    init_uart0();
    stdout = &OUTPUT; // printf가 동작할 수 있도록 stdout을 설정

    ds1302_main();

    while (1)
    {
```

```

        //dht11_main();
        //_delay_ms(1500);      // DHT11 안정화 시간
    }
}
...

```

```
#ds1302.c
```

```

/*
 * ds1301.c
 *
 * Created: 2026-06-26 오후 2:42:13
 * Author: kccistc
 */

```

```
#include "ds1302.h"
```

```
#include <avr/interrupt.h>
```

```
void init_ds1302(t_ds1302 *p);
```

```
void init_date_time(t_ds1302 *p);
```

```
void init_ddr_ds1302(void);
```

```
void init_gpio_ds1302(void);
```

```
void ds1302_main(void);
```

```
void clock_ds1302(void);
```

```
void read_time_ds1302(t_ds1302 *p);
```

```
void read_date_ds1302(t_ds1302 *p);
```

```
void tx_ds1302(uint8_t data);
```

```
void rx_1302(uint8_t *pdata8bits);
```

```
void write_ds1302(uint8_t addr, uint8_t data);
```

```
uint8_t read_ds1302(uint8_t addr);
```

```
uint8_t dec2bcd(uint8_t data);
```

```
uint8_t bcd2dec(uint8_t data);
```

```
extern void pc_command_processing(t_ds1302 *p);
```

```
void ds1302_main(void)
```

```
{
```

```
    t_ds1302 rtc;
```

```
    init_date_time(&rtc);
```

```
    init_ddr_ds1302();
```

```

init_gpio_ds1302();           // all LOW로 설정
init_ds1302(&rtc);
sei();    // UART 인터럽트 활성화

while (1)
{
    pc_command_processing(&rtc);

    // 1. read time
    read_time_ds1302(&rtc);

    // 2. read date
    read_date_ds1302(&rtc);

    // 3. printf date & time
    printf("DATE: %d-%d-%d ", rtc.year, rtc.month, rtc.date);
    printf("TIME: %d-%d-%d\n", rtc.hour, rtc.minutes, rtc.seconds);

    // 4. delay_ms(1000);
    _delay_ms(1000);
}

};

void read_time_ds1302(t_ds1302 *p)
{
    p->seconds = read_ds1302(ADDR_SECONDS);
    p->minutes = read_ds1302(ADDR_MINUTES);
    p->hour    = read_ds1302(ADDR_HOUR);
}

void read_date_ds1302(t_ds1302 *p)
{
    p->date  = read_ds1302(ADDR_DATE);
    p->month = read_ds1302(ADDR_MONTH);
    p->year  = read_ds1302(ADDR_YEAR);
}

uint8_t read_ds1302(uint8_t addr)
{
    uint8_t data8bits = 0;    // 1bit씩 읽어서 답을 변수

```

```

// 4. CE low
// 5. return(bcd to dec)

// 1. CE low --> High
DS1302_RST_PORT |= 1 << DS1302_RST;

// 2. ADDR 전송
tx_ds1302(addr+1);          // read addr

// 3. DATA 읽어들이
rx_1302(&data8bits);

// 4. CE HIGH ---> LOW
DS1302_RST_PORT &= ~(1 << DS1302_RST);

return bcd2dec(data8bits);
}

void rx_1302(uint8_t *pdata8bits)
{
    uint8_t temp=0;

    // 1. 입력 모드로 설정
    DS1302_DAT_DDR &= ~(1 << DS1302_DAT);          // write mode
    _delay_us(2);

    // LSB로부터 차례로
    for (int i=0; i<8; i++)
    {
        if (DS1302_DAT_PIN & (1 << DS1302_DAT))
        {
            temp |= 1 << i;          // 1의 조건만 set
        }

        // 3. CLK을 high -> low
        if (i != 7)                  // 마지막 비트를 읽을 때는 clk
            clock_ds1302();          을 보내주지 않는다.

        *pdata8bits = temp;
    }
};

```

```

void init_ds1302(t_ds1302 *p)
{
    write_ds1302(ADDR_WRITEPROTECTED, 0x00);

    write_ds1302(ADDR_SECONDS, p->seconds);
    write_ds1302(ADDR_MINUTES, p->minutes);
    write_ds1302(ADDR_HOUR, p->hour);
    write_ds1302(ADDR_DATE, p->date);
    write_ds1302(ADDR_MONTH, p->month);
    write_ds1302(ADDR_DAYOFWEEK, p->dayofweek);
    write_ds1302(ADDR_YEAR, p->year);
}

```

```

// decimal --> bcd
// ex) 25
//      dec          bcd
//      00011001    0010 0101

```

```

uint8_t dec2bcd(uint8_t data)
{
    uint8_t high, low;

    high = (data/10) << 4; // high nibble에 위치
    low = data % 10;

    return (high+low);
}

```

```

// bcd --> decimal
// 1. 입력 bcd
// ex. 26년의 bcd
// 7654 3210
// 0010 0110
//      2          6
// x10   x1
// =====
// 26

```

```

uint8_t bcd2dec(uint8_t data)
{
    uint8_t high, low;

```

```

        low = data & 0x0f;
        high = (data >> 4) * 10;

        return (high + low);
    }

void write_ds1302(uint8_t addr, uint8_t data)
{
    // 1. CE low --> High
    DS1302_RST_PORT |= 1 << DS1302_RST;

    // 2. ADDR 전송
    tx_ds1302(addr);

    // 3. DATA 전송
    tx_ds1302(dec2bcd(data));

    // 4. CE HIGH ---> LOW
    DS1302_RST_PORT &= ~(1 << DS1302_RST);
}

void tx_ds1302(uint8_t data)
{
    // 1. 출력 모드로 설정
    DS1302_DAT_DDR |= 1 << DS1302_DAT;           // write mode
    // ex) 0x80
    //           M           L
    //           1000 000
    for (int i=0; i<8; i++)
    {
        // 2. 1 bit를 전송
        if (data & (1 << i))
            DS1302_DAT_PORT |= 1 << DS1302_DAT;           // 1
        else DS1302_DAT_PORT &= ~(1 << DS1302_DAT);       // 0

        // 3. CLK을 high -> low
        clock_ds1302();
    }
}

void clock_ds1302(void)

```

```

{
    // LOW ----> HIGH ----> LOW
    DS1302_CLK_PORT &= ~(1<<DS1302_CLK);
    _delay_us(1);
    DS1302_CLK_PORT |= 1<<DS1302_CLK;
    _delay_us(1);
    DS1302_CLK_PORT &= ~(1<<DS1302_CLK);
    _delay_us(1);
}

void init_ddr_ds1302(void)
{
    DDRF &= ~(1 << DS1302_CLK | 1 << DS1302_DAT | 1 << DS1302_RST);
    DDRF |= 1 << DS1302_CLK | 1 << DS1302_DAT | 1 << DS1302_RST;    // 출력
    력 모드로 설정
}

void init_gpio_ds1302(void)
{
    DS1302_CLK_PORT &= ~(1 << DS1302_CLK | 1 << DS1302_DAT | 1 <<
DS1302_RST);
    _delay_ms(2);
}

void init_date_time(t_ds1302 *p)
{
    p->year      = 26;
    p->month      = 06;
    p->date       = 29;
    p->dayofweek  = 6;
    p->hour       = 10;
    p->minutes    = 02;
    p->seconds    = 00;
}

```

```

#ds1302.h
/*
 * ds1302.h
 *
 * Created: 2026-06-26 오후 2:42:24
 * Author: kccistc
 */

#ifndef DS1302_H_
#define DS1302_H_

#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include <string.h>

#define DS1302_CLK_DDR      DDRF
#define DS1302_CLK_PORT PORTF

#define DS1302_DAT_DDR      DDRF
#define DS1302_DAT_PORT PORTF
#define DS1302_DAT_PIN  PINF  // read

#define DS1302_RST_DDR  DDRF
#define DS1302_RST_PORT PORTF

#define DS1302_CLK      0
#define DS1302_DAT      1
#define DS1302_RST      2

#define ADDR_SECONDS      0x80  // write만 define(read는 write+1 이므로)
#define ADDR_MINUTES      0x82
#define ADDR_HOUR          0x84
#define ADDR_DATE          0x86
#define ADDR_MONTH         0x88
#define ADDR_DAYOFWEEK     0x8a
#define ADDR_YEAR          0x8c
#define ADDR_WRITEPROTECTED 0x8e

typedef struct _ds1302
{

```

```
uint8_t seconds;
uint8_t minutes;
uint8_t hour;
uint8_t date;
uint8_t month;
uint8_t dayofweek;    // 1.sun 2.mon
uint8_t year;
uint8_t ampm;         // 1.pm 0.am
uint8_t hourmode;     // 0: 24 1: 12

} t_ds1302;

// t_ds1302 ds1302;

#endif /* DS1302_H_ */
```

```

#uart0.c
/*
 * uart0.c
 *
 * Created: 2026-06-16 오전 9:57:55
 * Author: kccistc
 */

#include "uart0.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "ds1302.h"
void init_uart0();
void UART0_transmit(uint8_t data);
void pc_command_processing(t_ds1302 *p);
extern void write_ds1302(uint8_t addr, uint8_t data);

extern int func_state;
extern void (*fp[]) ();
extern volatile char scm[50];

// p.278 표 12-3
// pc로부터 1byte 들어오면 자동으로 이곳에 진입
// ex) led_all_on\n 이면 11번 이곳으로 진입
ISR(USART0_RX_vect)
{
    // interrupt 쓰는 변수는 volatile로 해야함
    volatile uint8_t data;
    volatile static int i=0;

    // UDR0를 data에 복사하는 순간 UDR0의 내용은 없어지므로, 데이터가 '이동'한다고
    생각해도 무방함
    data = UDR0;

    if (data == '\n' || data == '\r')
    {
        if ((rear+1) % QUEUE_SIZE == front % QUEUE_SIZE)
        {
            return; // queue full 상태
        }
        rx_buff[rear][i] = '\0'; // 문장의 끝인 NULL을 넣는다.
        i = 0; // 다음 string
    }
}

```

을 저장하기 위해 i를 0으로 만든다.

```
        rear = (rear + 1) % QUEUE_SIZE; // 0~9
    }
    else
    {
        if ((rear+1) % QUEUE_SIZE == front % QUEUE_SIZE)
        {
            return; // queue full 상태
        }
        rx_buff[rear][i++] = data;
    }
}

/*
1. 전송속도 : 9600bps
2. start/stop 설정
3. RX(수신): interrupt 로 설정

*/
void init_uart0(void){
    UBRR0H = 0x00;
    UBRR0L = 207; // 9600bps, 표 8-9
    UCSR0A |= 1 << U2X0; // 2배속 설정
    // UART0 송신, 수신이 모두 가능하도록 RX Interrupt가 가능하도록 설정.
    UCSR0B |= 1 << RXEN0 | 1 << TXEN0 | 1 << RXCIE0; // RXCIE: RX complete
    interrupt enable
}

// UART0로 1byte 전송하는 함수
void UART0_transmit(uint8_t data)
{
    while( !(UCSR0A & (1 << UDRE0))); // no operation -> data가 송신중이면 송신이
    끝날 때까지 기다림

    UDR0 = data; // HW 전송 register에 data를 송신한다.
}

void pc_command_processing(t_ds1302 *p)
{
    if (front == rear) return;

    char cmd[QUEUE_LENGTH];
```

```

strcpy(cmd, (char*)rx_buff[front]);
front = (front + 1) % QUEUE_SIZE;

// "setrtc" + 12자리 숫자 확인
if (strncmp(cmd, "setrtc", 6) != 0 || strlen(cmd) != 18)
{
    printf("ERR: setrtc YYMMDDHHMMSS\n");
    return;
}

char *s = cmd + 6; // "260629095600" 시작 포인터

// 2자리씩 잘라서 파싱
char buf[3] = {0};

strcpy(buf, s+0, 2); p->year = atoi(buf);
strcpy(buf, s+2, 2); p->month = atoi(buf);
strcpy(buf, s+4, 2); p->date = atoi(buf);
strcpy(buf, s+6, 2); p->hour = atoi(buf);
strcpy(buf, s+8, 2); p->minutes = atoi(buf);
strcpy(buf, s+10, 2); p->seconds = atoi(buf);

// RTC에 쓰기
write_ds1302(ADDR_WRITEPROTECTED, 0x00);
write_ds1302(ADDR_YEAR, p->year);
write_ds1302(ADDR_MONTH, p->month);
write_ds1302(ADDR_DATE, p->date);
write_ds1302(ADDR_HOUR, p->hour);
write_ds1302(ADDR_MINUTES, p->minutes);
write_ds1302(ADDR_SECONDS, p->seconds);

printf("SET: 20%02d-%02d-%02d %02d:%02d:%02d\n",
p->year, p->month, p->date,
p->hour, p->minutes, p->seconds);
}

```

```
#uart0.h
/*
 * uart0.h
 *
 * Created: 2026-06-16 오전 9:58:08
 * Author: kccistc
 */

#ifndef UART0_H_
#define UART0_H_
#define F_CPU 16000000UL // 16MHz
#include <avr/io.h> // PORTA PORTB PORTD... IO관련 reg가 들어 있다.
#include <util/delay.h> // _delay_ms _delay_us 등
#include <avr/interrupt.h>
#include <string.h>

#define QUEUE_SIZE 10
#define QUEUE_LENGTH 80
volatile int rear=0; // interrupt에서 queue에 data를 저장하는 위치값
volatile int front=0; // pc_command_processing에서 가져가는 위치값
volatile unsigned char rx_buff[QUEUE_SIZE][QUEUE_LENGTH];

#endif /* UART0_H_ */
```